Name: Pragun Damani Net ID: pd2752 Assigned Seat: J-18

NYU, Tandon School of Engineering
CS-1134: Data Structures and Algorithms — Fall 2025

CS-1134 – Midterm Exam #1

Tuesday, October 21, 2025

a) **Duration**: 1 hour and 15 minutes.

b) **Page Usage**:
   - Write your answers clearly and concisely, in the spaces on the exam.
   - Do not write on the back of any page.
   - **Do not separate** the pages.
   - If you need extra space for an answer, use the extra page at the end of the exam and mark it clearly, so we can find it when we're grading.

c) **Writing Guidelines**:
   - Use pen only; if you must use a pencil, ensure it is dark enough for scanning.

d) **Closed Book Policy**: This is a closed book exam; calculators are not allowed.

e) **User Inputs**: You may assume that user inputs will be valid.

f) **Comments**: Comments are not necessary.

g) **Academic Integrity**: Any student found cheating will receive a zero for the exam.

h) **Timing**: Any student who continues writing after time has been called will receive a zero.

i) **Test Opening**: Do not open this test booklet until instructed to do so.

j) **Questions**: If you have a question, please ask the exam proctor.

Point Distribution

- Question 1 – 20 points
- Question 2 – 22 points
- Question 3 – 23 points
- Question 4 – 15 points
- Question 5 – 20 points

Name: _____ Net ID: _____

Question 1 (20 points)

a) Determine what is printed when the following code runs.

```
>>> def func(a):
        b = a
        a[1] = 99
        a = a + [100]
        b.append(7)
        print("Inside func: a = ", a)
        print("Inside func: b = ", b)

>>> x = [1, 2, 3]
>>> y = x
>>> z = x[ : ]
>>> func(x)
>>> print("Outside: x = ", x)
>>> print("Outside: y = ", y)
>>> print("Outside: z = ", z)
```

Output:

Inside func: a = 1, 99, 3, 100, 7

Inside func: b = 1, 99, 3, 100, 7

Outside: x = 1, 99, 3, 100, 7

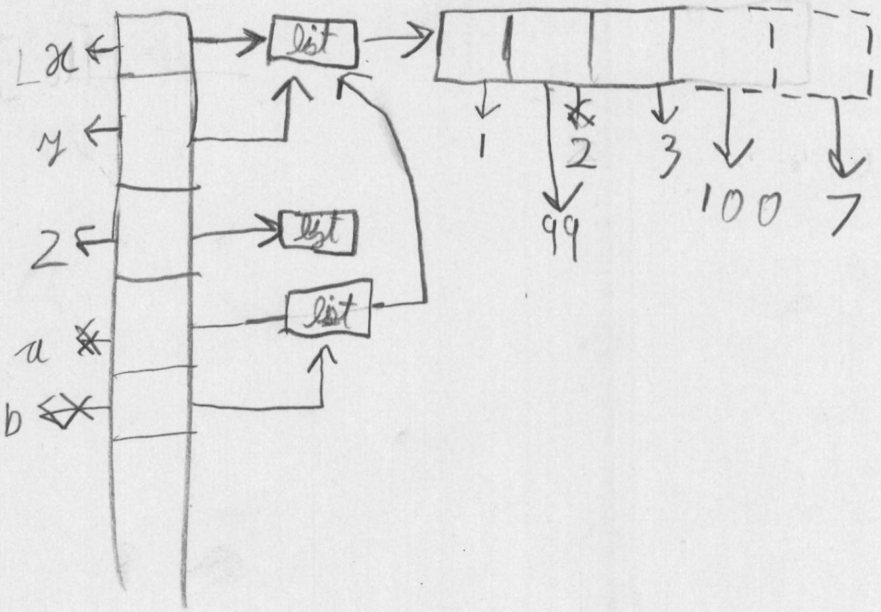
Outside: y = 1, 99, 3, 100, 7

Outside: z = 1, 99, 3, 100, 7

b) On the next page, draw the memory model of the **end-time** of the execution of the code snippet above, indicating how the lists are structured in memory.

Name: _____ Net ID: _____

Memory Model:



Name: _____ Net ID: _____

Question 2 (22 points)

a) What is printed when the following Python code is executed?

```
>>> def gen(n):
        curr = 0
        sum = 0
        while curr < n:
            curr = curr + 1
            sum += curr
            yield curr
        yield sum

>>> for num in gen(3):
        print(num)
```

Output:

1

2

3

6

Name: Brayan Net ID: pd2752

b) You are given the following function:

```
def func1(n):
    count = 0

    i = n
    while(i >= 1):
        for j in range(i):
            count += 1
        i //= 2

    return count
```

log(n) n i

What is the worst-case running time of `func1(n)`?
Give a short explanation of your answer.

$T(n) = \theta(\underline{(\log(n))^2})$

Since while loop runs log (n) times it creates log (n) values of j
∴ it takes $\lfloor \log(n) \rfloor \times \log(n)$

Name: ~~Prof~~ Jose Bragan Net ID: pd2752

c) You are given the following function:

```
def func2(n):
    count = 0

    for i in range(n·√n):
        for j in range(i):
            count += 1

    return count
```

What is the worst-case running time of `func2(n)`?
Give a short explanation of your answer.

$T(n) = \theta($ ~~n^2~~ n^3 $)$

① ~~Since~~ ~~count~~ content of loop 2 is constant
② loop 2 runs i operations
③ loop 1 ~~iterates~~ runs $n\sqrt{n}$ ~~times~~
∴ there are $n\sqrt{n}$ ~~operations~~ i operations
∴ thus cost is $(n\sqrt{n})^2 = n^3$

Name: Burgun Net ID: pd2752

Question 3 (23 points)

Implement the following function:

```
def remove_consecutive_duplicates(lst)
```

The above function gets a list of integers, `lst`. When called, it should **mutate** `lst` so that any consecutive duplicates collapse down to a single occurrence.

For example, if `lst = [2, 2, 2, 3, 3, 1, 1, 1, 2, 2]`, after calling `remove_consecutive_duplicates(lst)`, `lst` should be the following 4-element list: `[2, 3, 1, 2]`.

Your implementation should provide the following behavior:

```
>>> lst = [2, 2, 2, 3, 3, 1, 1, 1, 2, 2]
>>> remove_consecutive_duplicates(lst)
>>> lst
[2, 3, 1, 2]
```

Implementation requirements:

1. Your implementation should run in **linear time**. That is, if *n* is the number of elements in `lst`, your function should run in $\theta(n)$.
2. You are only allowed to use $\theta(1)$ **additional memory**.
   That is, you are allowed to use any number of variables that take constant space (`int`, `double`, etc). You are **not allowed** to use any additional data collections (such as `list`, `tuple`, `string`, `dictionary`, `set`, etc).

```
ind = 0
~~end = len(lst) - 1~~
while i < e:
    if l[i] == l[i+1]:
        l[i+1], l[e] = l[e], l[i+1]
        l.pop()
        ind += 1
    else:
        i += 1
```

Name: Prav Prayun Net ID: pd2752

```
def remove_consecutive_duplicates(lst):
    ind = 0  # index
    ~~end = len(lst)~~
    while ind < ~~len~~(lst) - 1:
        if lst[ind] == lst[ind+1]:
            lst[ind+1], lst[len(lst)-1], = lst[len(lst)-1], ~~lst[ind]~~
, lst[ind]
            ~~lst[len(lst)-1~~ lst.pop()
        ~~else~~:
            ~~ind += 1~~
            ~~ind += 1~~
        else:
            ind += 1
```

Name: Pr Pragm Net ID: pd2752

Question 4 (15 points)

You are given the following recursive function:

```
def mystery(s):
    if(len(s) <= 1):
        return s
    else:
        rest_res = mystery(s[1 : ])
        return s[0] + rest_res + s[0]
```

m a d a m

1) What would be returned from the call `mystery('mad')`:

madam

2) Give a meaningful name to `mystery` (A name that describes what this function computes).

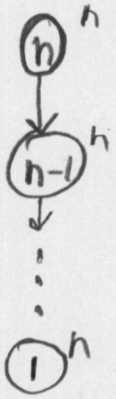
palindrome-creator(s):

3) Assuming `s` is a string with n characters, analyze the worst-case running time of the call `mystery(s)`:
   i) Draw the recursion tree (showing the recursive calls hierarchy, and their local cost).
   ii) Conclude the total (asymptotic) running time of the function.

Note: Write your answers in the next page.

Name: Program Net ID: _____

i. Draw the **recursion tree** for `mystery(s)`



ii. **Conclude the running time** of `mystery(s)`

$T(n) = \Theta($ n^2 $)$

Calculations:

~~At~~ $n-1$ recursive calls ~~are~~ are made that ~~m~~ slice array taking $n-1$ time

$(n-1)^2 = \Theta(n^2)$

Name: _____ Net ID: _____

Question 5 (20 points)

Only customers whose **order-ID is a multiple of k** qualify for a gold voucher. You're given a list of order-IDs in the exact arrival order. Your task is to produce a **new list** that keeps **only** the order-IDs that are divisible by k, **preserving their original relative order**.

Give a **recursive** implementation for the function:

```
def select_gold_voucher_orders(orders, k, low, high)
```

This function receives a list of order-IDs, `orders`, and a positive integer k. In addition, the function is called with two indices `low` and `high` (with `low` ≤ `high`), which define the range of elements to consider within the list.
When called, it should return a **new list** containing all the order-IDs found in `orders` from index `low` to index `high` (inclusive). The original order should be preserved.

For example, the call:
`select_gold_voucher_orders([12, 7, 9, 18, 5], 3, 0, 4)`,
should return `[12, 9, 18]`.

Implementation requirements:

i. Your function **must be recursive**.
ii. You are **not allowed** to:
- Define a helper function.
- Add parameters to the function's header line
- Set default values to the parameter.
- Use global variables

iii. Your function should run in **worst case linear time**.
That is, if n is the size of the range from index `low` to index `high`, calling `select_gold_voucher_orders(orders, k, low, high)` will run in $\theta(n)$.

Note: Write your implementation on the next page.

flag1 = low % k == 0
flag2 = high % k == 0
if flag1 and flag2:
  low+=1
  high-=1
if flag1 and not flag2:
  low+=1
  lst.pop()
  high-=1

if not flag1 and flag2:
  high-=1
if not flag1 and not flag2:
  lst.pop()
  high-=1

Name: _____ Net ID: _____

```
def select_gold_voucher_orders(orders, k, low, high):
```

```
1   lst, flag1, flag2 = orders, (low % k == 0), (high % k == 0)
2   if flag1 and flag2:
3       low += 1
4       high -= 1
5       return select_gold_voucher_orders(lst, k, low, high)
6   elif flag1 and not flag2:
7       low += 1
8       lst.pop()
9       return select_gold_voucher_orders(lst, k, low, high)
10  elif not flag1 and flag2:
11      high -= 1
12  elif not flag1 and not flag2:
13      lst.pop()
14      high -= 1
15  if low > high:
16      return lst
```

continued on last page

label 5, 16

12, 7, 9, 18, 8

l h

l h

Name: _____ Net ID: _____

EXTRA PAGE IF NEEDED

Note question numbers of any questions or part of questions that you are answering here.
Also, write "ANSWER IS ON LAST PAGE" near the space provided for the answer.

Q5

```
if high == low and not flag1:
    lst.pop(high)
```